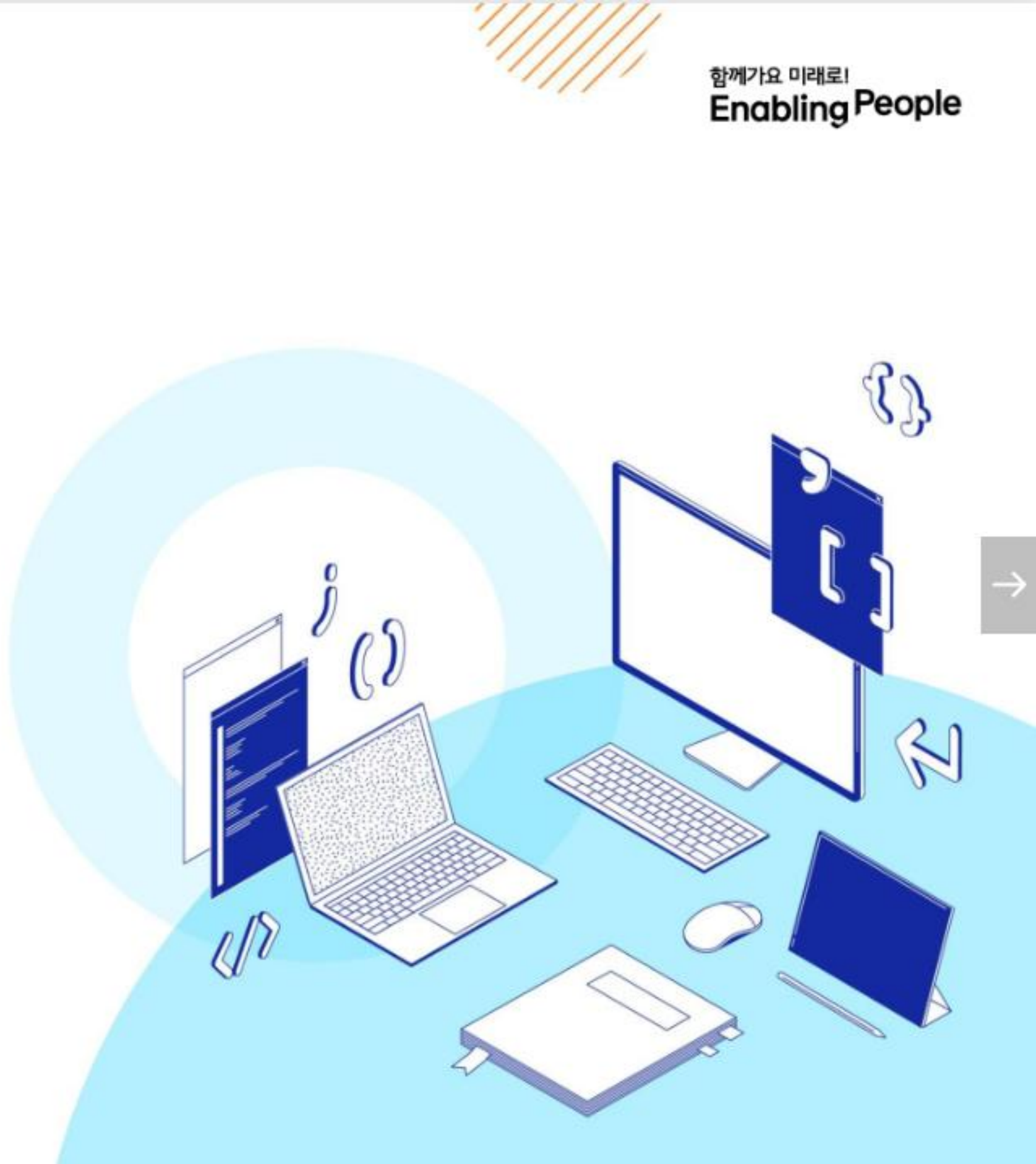


SSAFY

DB

SQL 02

Python



목 차

Managing Tables

- Create a table
- Modifying table fields
- Delete a table

Modifying Data

- Insert data
- Update data
- Delete data

Multi table queries

- Join
- Joining tables

목 차

참고

- 타입 선호도
- NOT NULL
- 날짜와 시간



"나만의 작은 도서관, 직접 만들어 볼까요?"

- 집에 있는 책을 정리해서 나만의 도서관을 만든다고 상상해 보세요!
 - 이제 어떤 책이 있는지, 누가 빌려갔는지, 언제 샀는지까지 정리해보려면 어떻게 해야 할까요?

1. 책장 하나를 [] 문으로 만들고,
2. 책 정보를 [] 문으로 채워 넣습니다.
3. "2020년 이후에 산 책"만 [] 문으로 골라볼 수도 있죠.

- SQL은 데이터를 정리하는 책장 같은 도구입니다.
- 원하는 데이터를 빠르게 찾을 수 있도록 도와줍니다.
- 스스로 데이터를 만들고, 저장하고, 찾아보는 과정을 직접 체험할 수 있습니다.

학습 목표

- ✓ SQL의 기본 문법 구조를 이해한다.
- ✓ CREATE TABLE 문을 사용하여 테이블을 생성할 수 있다.
- ✓ INSERT 문을 활용하여 테이블에 데이터를 삽입할 수 있다.
- ✓ SELECT 문으로 테이블에서 원하는 데이터를 조회할 수 있다.
- ✓ ALTER TABLE 문을 통해 테이블 구조를 수정할 수 있다.
- ✓ DROP TABLE 문으로 테이블을 삭제할 수 있다.
- ✓ UPDATE와 DELETE 문으로 데이터를 수정하거나 삭제할 수 있다.
- ✓ INNER JOIN과 LEFT JOIN의 차이점을 이해하고 사용할 수 있다.
- ✓ 제약 조건(CONSTRAINT)의 의미를 알고 테이블 생성 시 적용할 수 있다.

Managing Tables

SQL Statements 유형

유형	역할	SQL 키워드
DDL (Data Definition Language)	데이터의 기본 구조 및 형식 변경	CREATE DROP ALTER
DQL (Data Query Language)	데이터 검색	SELECT
DML (Data Manipulation Language)	데이터 조작 (추가, 수정, 삭제)	INSERT UPDATE DELETE
DCL (Data Control Language)	데이터 및 작업에 대한 사용자 권한 제어	COMMIT ROLLBACK GRANT REVOKE

Create a table

CREATE TABLE

CREATE TABLE syntax

```
CREATE TABLE table_name (  
    column_1 data_type constraints,  
    column_2 data_type constraints,  
    ...,  
);
```

- 각 필드에 적용할 데이터 타입 작성
- 테이블 및 필드에 대한 제약조건(constraints) 작성



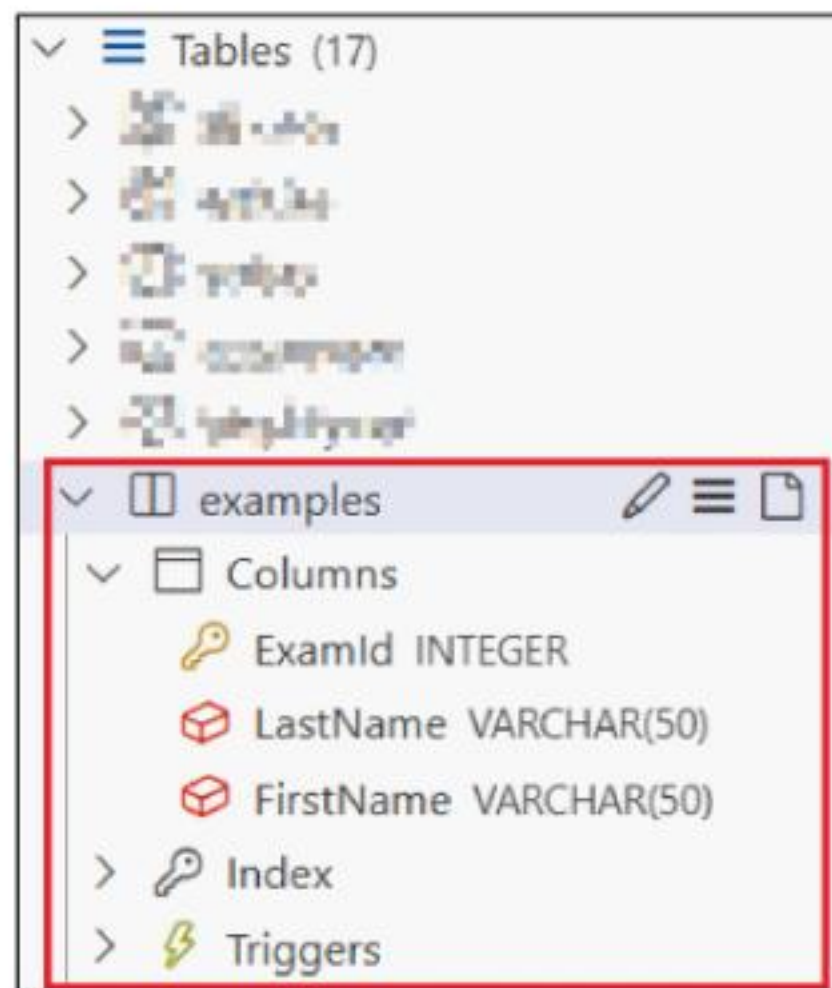
CREATE TABLE statement
테이블 생성

<개념1_DB_SQL 2_CREATE TABLE statement 정의>

CREATE TABLE 활용

- examples 테이블 생성 및 확인

```
CREATE TABLE examples (  
    ExamId INTEGER PRIMARY KEY AUTOINCREMENT,  
    LastName VARCHAR(50) NOT NULL,  
    FirstName VARCHAR(50) NOT NULL  
);
```



<그림1_DB_SQL 2_examples 테이블 생성>

PRAGMA

- 테이블 schema(구조) 확인

```
PRAGMA table_info('examples');
```

Q	cid	name	type	notnull	dflt_value	pk
>	0	ExamId	INTEGER	0	(NULL)	1
>	1	LastName	VARCHAR(50)	1	(NULL)	0
>	2	FirstName	VARCHAR(50)	1	(NULL)	0

<그림2_DB_SQL 2_examples 테이블 구조 확인>

TIP

“cid”

- Column ID를 의미하며 각 컬럼의 고유한 식별자를 나타내는 정수 값
- 직접 사용하지 않으며 PRAGMA 명령과 같은 메타데이터 조회에서 출력 값으로 활용됨

CREATE TABLE Statement 구성 (1/3)

- 데이터 타입

```
CREATE TABLE examples (  
    ExamId INTEGER PRIMARY KEY AUTOINCREMENT,  
    LastName VARCHAR(50) NOT NULL,  
    FirstName VARCHAR(50) NOT NULL  
);
```

CREATE TABLE Statement 구성 (2/3)

- 제약 조건

```
CREATE TABLE examples (  
    ExamId INTEGER PRIMARY KEY AUTOINCREMENT,  
    LastName VARCHAR(50) NOT NULL,  
    FirstName VARCHAR(50) NOT NULL  
);
```

CREATE TABLE Statement 구성 (3/3)

- AUTOINCREMENT 키워드

```
CREATE TABLE examples (  
    ExamId INTEGER PRIMARY KEY AUTOINCREMENT,  
    LastName VARCHAR(50) NOT NULL,  
    FirstName VARCHAR(50) NOT NULL  
);
```

SQLite 데이터 타입

- NULL
 - 아무런 값도 포함하지 않음을 나타냄
- TEXT
 - 문자열
- INTEGER
 - 정수
- REAL
 - 부동 소수점
- BLOB
 - 이미지, 동영상, 문서 등의 바이너리 데이터

제약 조건이란?

Constraints

제약 조건

테이블의 필드에 적용되는 규칙 또는 제한 사항

- 데이터의 무결성을 유지하고 데이터베이스의 일관성을 보장

대표 제약 조건 3가지

- PRIMARY KEY
 - 해당 필드를 기본 키로 지정
 - ※ INTEGER 타입에만 적용되며 INT, BIGINT 등과 같은 다른 정수 유형은 적용되지 않음
- NOT NULL
 - 해당 필드에 NULL 값을 허용하지 않도록 지정
- FOREIGN KEY
 - 다른 테이블과의 외래 키 관계를 정의



Constraints

테이블의 필드에 적용되는
규칙 또는 제한 사항

<개념2_DB_SQL 2_제약 조건 정의>

AUTOINCREMENT 특징

- 필드의 자동 증가를 나타내는 특수한 키워드
- 주로 primary key 필드에 적용
- INTEGER PRIMARY KEY AUTOINCREMENT가 작성된 필드는 항상 새로운 레코드에 대해 이전 최대 값보다 큰 값을 할당
- 삭제된 값은 무시되며 재사용할 수 없게 됨



AUTOINCREMENT keyword

자동으로 고유한 정수 값을
생성하고 할당하는 필드 속성

<개념3_DB_SQL 2_AUTORINCREMENT 키워드 정의>

Modifying table fields

ALTER TABLE

ALTER TABLE 역할

명령어	역할
ALTER TABLE ADD COLUMN	필드 추가
ALTER TABLE RENAME COLUMN	필드 이름 변경
ALTER TABLE DROP COLUMN	필드 삭제
ALTER TABLE RENAME TO	테이블 이름 변경

<표2_DB_SQL 2 ALTER TABLE 명령어 활용 예시>



ALTER TABLE statement
테이블 및 필드 조작

<개념4_DB_SQL 2 ALTER TABLE 문 정의>

1. ALTER TABLE ADD COLUMN syntax

```
ALTER TABLE  
    table_name  
ADD COLUMN  
    column_definition;
```

- ADD COLUMN 키워드 이후 추가하고자 하는 새 필드 이름과 데이터 타입 및 제약 조건 작성

※ 단, 추가하고자 하는 필드에 NOT NULL 제약조건이 있을 경우 NULL이 아닌 기본 값 설정 필요



ALTER TABLE statement
테이블 및 필드 조작

<개념4_DB_SQL 2 ALTER TABLE 문 정의>



ADD COLUMN
필드 추가

<개념5_DB_SQL 2 ADD COLUMN 정의>

ALTER TABLE ADD COLUMN 활용 1

- examples 테이블에 다음 조건에 맞는 Country 필드 추가

Q	cid	name	type	notnull	dflt_value	pk
>	0	ExamId	INTEGER	0	(NULL)	1
>	1	LastName	VARCHAR(50)	1	(NULL)	0
>	2	FirstName	VARCHAR(50)	1	(NULL)	0
>	3	Country	VARCHAR(100)	1	'default value'	0

<그림3_DB_SQL2_컬럼 추가 결과>

- ※ 테이블 생성시 정의한 필드는 기본 값이 없어도 NOT NULL 제약조건으로 생성됨
- ※ 내부적으로 Default value는 NULL로 설정됨

ALTER TABLE ADD COLUMN 활용 1

- examples 테이블에 다음 조건에 맞는 Country 필드 추가

```
ALTER TABLE
  examples
ADD COLUMN
  Country VARCHAR(100) NOT NULL DEFAULT 'default value';
```

- ※ 쿼리 실행 후 “PRAGMA table_info('examples');”를 통해 결과 확인

ALTER TABLE ADD COLUMN 활용 2

- examples 테이블에 다음 조건에 맞는 Age, Address 필드 추가

Q	cid	name	type	notnull	dflt_value	pk
>	0	ExamId	INTEGER	0	(NULL)	1
>	1	LastName	VARCHAR(50)	1	(NULL)	0
>	2	FirstName	VARCHAR(50)	1	(NULL)	0
>	3	Country	VARCHAR(100)	1	'default value'	0
>	4	Age	INTEGER	1	0	0
>	5	Address	VARCHAR(100)	1	'default value'	0

<그림4_DB_SQL 2_age, address 필드 추가>

ALTER TABLE ADD COLUMN 활용 2

- examples 테이블에 다음 조건에 맞는 Age, Address 필드 추가

```
ALTER TABLE examples  
ADD COLUMN Age INTEGER NOT NULL DEFAULT 0;
```

```
ALTER TABLE examples  
ADD COLUMN Address VARCHAR(100) NOT NULL DEFAULT 'default value';
```

- ※ SQLite는 단일 문을 사용하여 한번에 여러 필드를 추가할 수 없음
- ※ 쿼리 실행 후 "PRAGMA table_info('examples');"를 통해 결과 확인

2. ALTER TABLE RENAME COLUMN syntax

```
ALTER TABLE  
    table_name  
RENAME COLUMN  
    current_name TO new_name;
```

- RENAME COLUMN 키워드 뒤에 이름을 바꾸려는 필드의 이름을 지정하고
TO 키워드 뒤에 새 이름을 지정



ALTER TABLE statement
테이블 및 필드 조작

<개념4_DB_SQL 2 ALTER TABLE 문 정의>



RENAME COLUMN
필드명 변경

<개념6_DB_SQL 2 RENAME COLUMN 정의>

ALTER TABLE RENAME COLUMN 활용 1

- examples 테이블 Address 필드의 이름을 PostCode로 변경

Q	cid	name	type	notnull	dflt_value	pk
>	0	ExamId	INTEGER	0	(NULL)	1
>	1	LastName	VARCHAR(50)	1	(NULL)	0
>	2	FirstName	VARCHAR(50)	1	(NULL)	0
>	3	Country	VARCHAR(100)	1	'default value'	0
>	4	Age	INTEGER	1	0	0
>	5	PostCode	VARCHAR(100)	1	'default value'	0

<그림5_DB_SQL2_필드명 변경 예시>

ALTER TABLE RENAME COLUMN 활용 1

- examples 테이블 Address 필드의 이름을 PostCode로 변경

```
ALTER TABLE examples  
RENAME COLUMN Address TO PostCode;
```

- ※ 쿼리 실행 후 “PRAGMA table_info('examples');”를 통해 결과 확인

3. ALTER TABLE DROP COLUMN syntax

```
ALTER TABLE  
    table_name  
DROP COLUMN  
    column_name;
```

- DROP COLUMN 키워드 뒤에 삭제 할 필드 이름 지정



ALTER TABLE statement
테이블 및 필드 조작

<개념4_DB_SQL 2 ALTER TABLE 문 정의>



DROP COLUMN
지정 필드 삭제

<개념7_DB_SQL 2 DROP COLUMN 정의>

ALTER TABLE DROP COLUMN 활용

- examples 테이블의 PostCode 필드를 삭제

Q	cid	name	type	notnull	dflt_value	pk
>	0	ExamId	INTEGER	0	(NULL)	1
>	1	LastName	VARCHAR(50)	1	(NULL)	0
>	2	FirstName	VARCHAR(50)	1	(NULL)	0
>	3	Country	VARCHAR(100)	1	'default value'	0
>	4	Age	INTEGER	1	0	0

<그림6_DB_SQL 2_필드 삭제 예시>

ALTER TABLE DROP COLUMN 활용

- examples 테이블의 PostCode 필드를 삭제

```
ALTER TABLE examples  
DROP COLUMN PostCode;
```

- ※ 쿼리 실행 후 “PRAGMA table_info('examples');”를 통해 결과 확인

4. ALTER TABLE RENAME TO syntax

```
ALTER TABLE  
    table_name  
RENAME TO  
    new_table_name;
```

- RENAME TO 키워드 뒤에 새로운 테이블 이름 지정



ALTER TABLE statement
테이블 및 필드 조작

<개념4_DB_SQL 2 ALTER TABLE 문 정의>

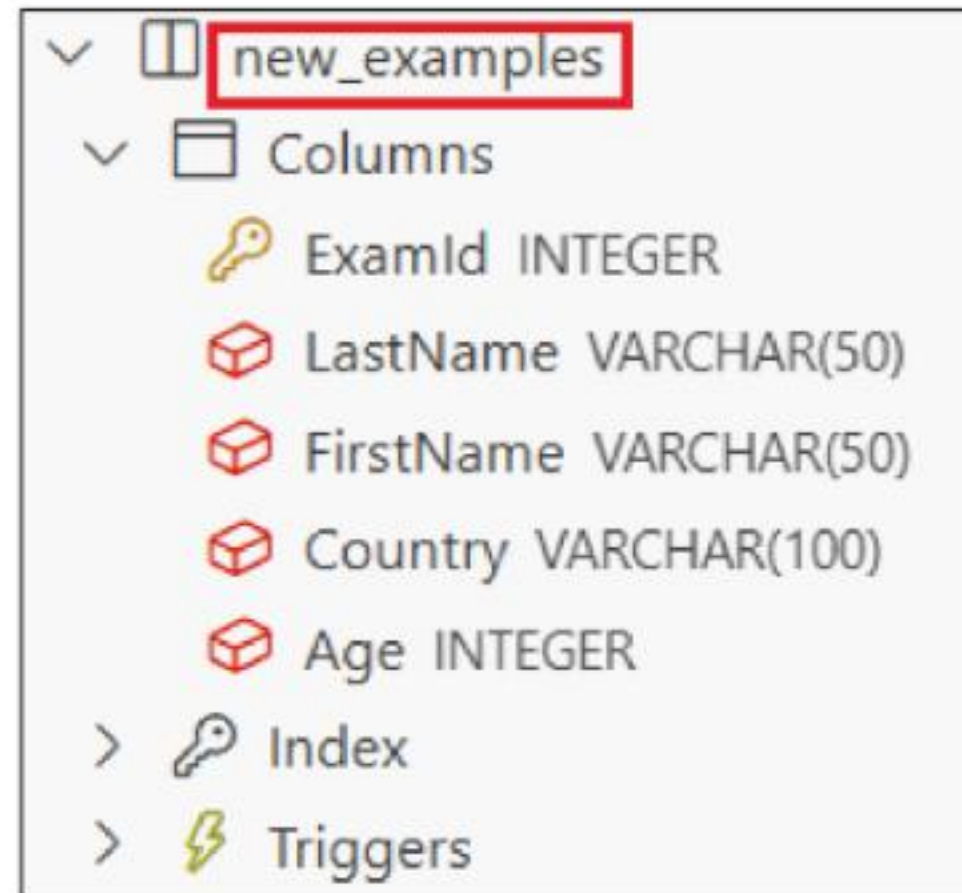
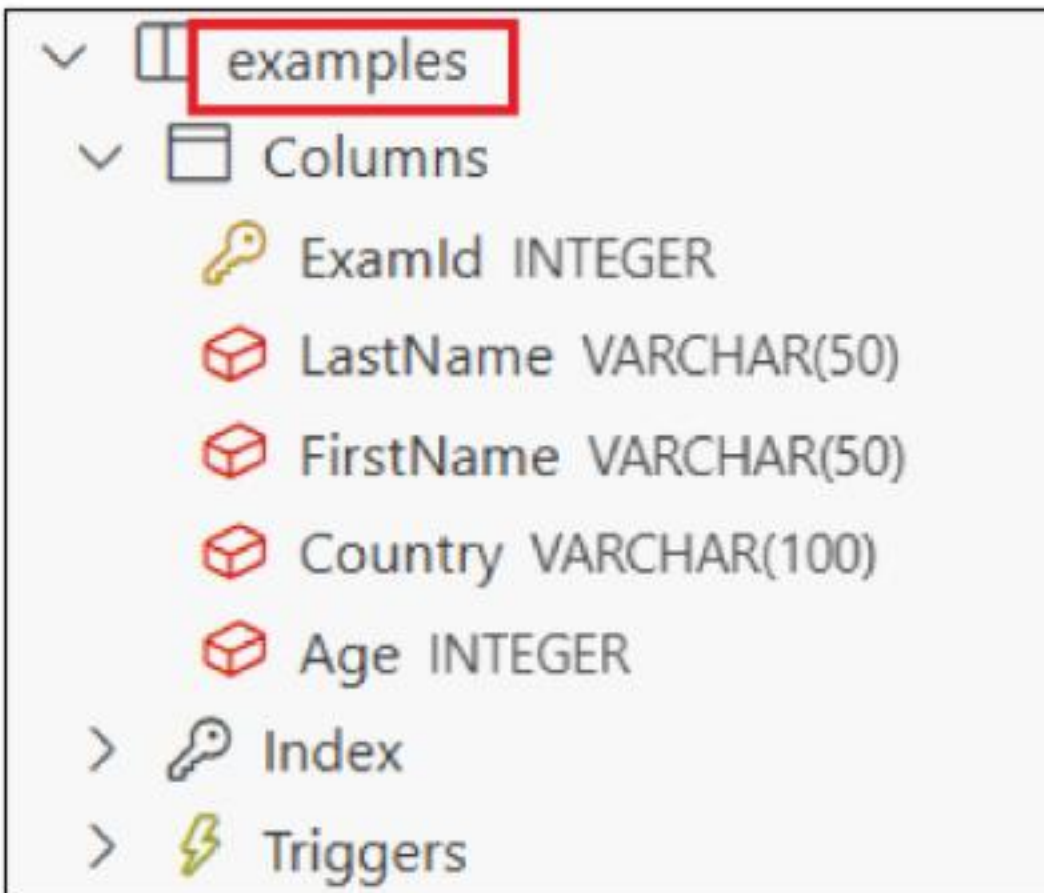


RENAME TO
테이블 명 수정

<개념8_DB_SQL 2 RENAME TO 정의>

ALTER TABLE RENAME TO 활용

- examples 테이블 이름을 new_examples로 변경



<그림7_DB_SQL 2_테이블명 변경 예시>

ALTER TABLE RENAME TO 활용

- examples 테이블 이름을 new_examples로 변경

```
ALTER TABLE examples  
RENAME TO new_examples;
```

Delete a table

DROP TABLE

DROP TABLE syntax

```
DROP TABLE table_name;
```

- DROP TABLE statement 이후 삭제할 테이블 이름 작성



DROP TABLE statement
테이블 삭제

<개념9_DB_SQL 2_DROP TABLE 문 정의>

DROP TABLE 활용

- new_examples 테이블 삭제

```
DROP TABLE new_examples;
```

Modifying Data

SQL Statements 유형

유형	역할	SQL 키워드
DDL (Data Definition Language)	데이터의 기본 구조 및 형식 변경	CREATE DROP ALTER
DQL (Data Query Language)	데이터 검색	SELECT
DML (Data Manipulation Language)	데이터 조작 (추가, 수정, 삭제)	INSERT UPDATE DELETE
DCL (Data Control Language)	데이터 및 작업에 대한 사용자 권한 제어	COMMIT ROLLBACK GRANT REVOKE

Insert data

INSERT

사전 준비 (1/2)

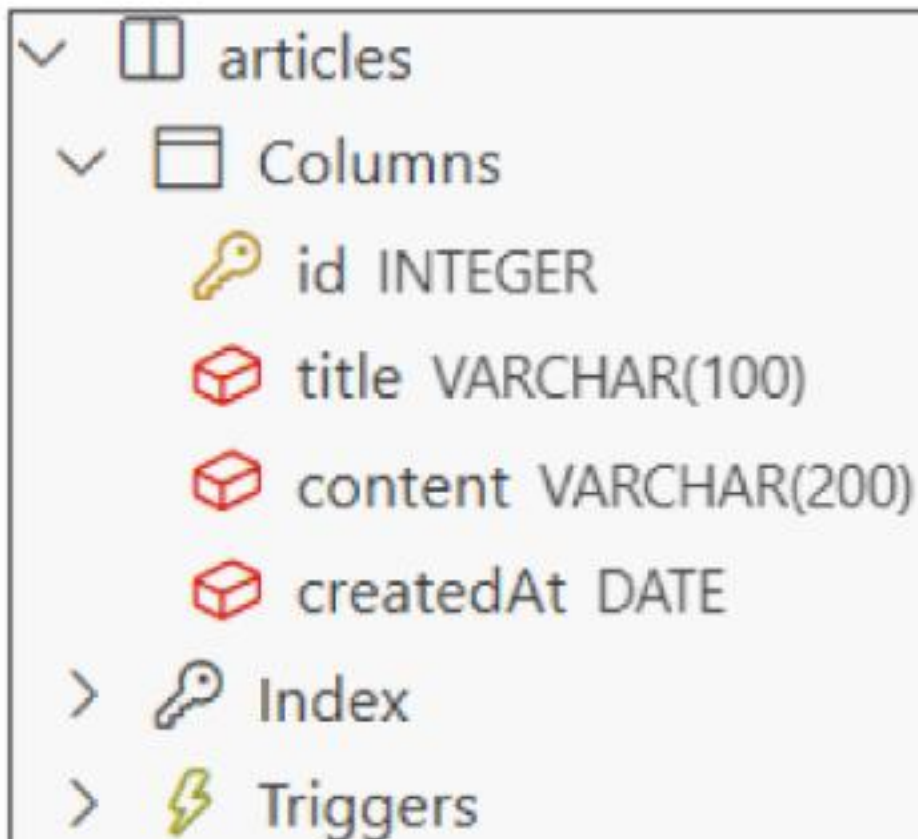
- 생성하려는 테이블과 동일한 이름을 가진 테이블이 존재하는지 확인
- 불필요한 테이블이라면 DROP TABLE 명령어로 테이블 삭제
- 정상적으로 삭제되었는지 PRAGMA 활용하여 체크

```
SELECT * FROM articles;  
DROP TABLE articles;  
PRAGMA table_info('articles');
```

사전 준비 (2/2)

- 실습 테이블 생성

```
CREATE TABLE articles (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title VARCHAR(100) NOT NULL,  
  content VARCHAR(200) NOT NULL,  
  createdAt DATE NOT NULL  
);
```



<그림9_DB_SQL 2_articles 테이블 생성>

INSERT syntax

```
INSERT INTO table_name (c1, c2, ...)
VALUES (v1, v2, ...);
```

- INSERT INTO 절 다음에 테이블 이름과 괄호 안에 필드 목록 작성
- VALUES 키워드 다음 괄호 안에 해당 필드에 삽입할 값 목록 작성



INSERT syntax
테이블 레코드 삽입

<개념10_DB_SQL 2_INSERT 정의>

INSERT 활용 1

- articles 테이블에 다음과 같은 데이터 입력

	 id INTEGER	* title VARCHAR(100)	* content VARCHAR(200)	* createdAt DATE
>	1	hello	world	2000-01-01

<그림10_DB_SQL 2_첫 번째 데이터 삽입 예시>

INSERT 활용 1

- articles 테이블에 다음과 같은 데이터 입력

```
INSERT INTO
  articles (title, content, createdAt)
VALUES
  ('hello', 'world', '2000-01-01');
```

- ※ 쿼리 실행 후 "SELECT * FROM articles;"를 통해 결과 확인

INSERT 활용 2

- articles 테이블에 다음과 같은 데이터 **추가** 입력

	 id INTEGER	* title VARCHAR(100)	* content VARCHAR(200)	* createdAt DATE
>	1	hello	world	2000-01-01
>	2	title1	content1	1900-01-01
>	3	title2	content2	1800-01-01
>	4	title3	content3	1700-01-01

<그림11_DB_SQL 2_여러 데이터 동시 삽입 예시>

INSERT 활용 2

- articles 테이블에 다음과 같은 데이터 **추가** 입력

```
INSERT INTO
  articles (title, content, createdAt)
VALUES
  ('title1', 'content1', '1900-01-01'),
  ('title2', 'content2', '1800-01-01'),
  ('title3', 'content3', '1700-01-01');
```

- ※ 쿼리 실행 후 "SELECT * FROM articles;"를 통해 결과 확인

INSERT 활용 3

- **DATE 함수**를 사용해 articles 테이블에 다음과 같은 데이터 추가 입력

	 id INTEGER	* title VARCHAR(100)	* content VARCHAR(200)	* createdAt DATE
>	1	hello	world	2000-01-01
>	2	title1	content1	1900-01-01
>	3	title2	content2	1800-01-01
>	4	title3	content3	1700-01-01
>	5	mytitle	mycontent	2025-07-29

<그림12_DB_SQL 2_DATE 함수를 활용한 데이터 삽입 예시>

INSERT 활용 3

- **DATE 함수**를 사용해 articles 테이블에 다음과 같은 데이터 추가 입력

```
INSERT INTO
  articles (title, content, createdAt)
VALUES
  ('mytitle', 'mycontent', DATE());
```

- ※ 쿼리 실행 후 "SELECT * FROM articles;"를 통해 결과 확인

Update data

UPDATE

UPDATE syntax

```
UPDATE table_name  
SET column_name = expression,  
[WHERE  
    condition];
```

- SET 절 다음에 수정할 필드와 새 값을 지정
- WHERE 절에서 수정할 레코드를 지정하는 조건 작성
- WHERE 절을 작성하지 않으면 모든 레코드를 수정



UPDATE statement
테이블 레코드 수정

<개념11_DB_SQL 2_UPDATE 정의>

UPDATE 활용 1

- articles 테이블 1번 레코드의 title 필드 값을 'update Title'로 변경

	id INTEGER	* title VARCHAR(100)	* content VARCHAR(200)	* createdAt DATE
>	1	update Title	world	2000-01-01
>	2	title1	content1	1900-01-01
>	3	title2	content2	1800-01-01
>	4	title3	content3	1700-01-01
>	5	mytitle	mycontent	2025-07-29

<그림13_DB_SQL 2_1번 레코드 title 수정 예시>

UPDATE 활용 1

- articles 테이블 1번 레코드의 title 필드 값을 'update Title'로 변경

```
UPDATE
  articles
SET
  title = 'update Title'
WHERE
  id = 1;
```

- ※ 쿼리 실행 후 "SELECT * FROM articles;"를 통해 결과 확인

UPDATE 활용 2

- articles 테이블 2번 레코드의 title, content 필드 값을 각각 'update Title', 'update Content' 로 변경

	Q	id INTEGER	* title VARCHAR(100)	* content VARCHAR(200)	* createdAt DATE
>		1	update Title	world	2000-01-01
>		2	update Title	update Content	1900-01-01
>		3	title2	content2	1800-01-01
>		4	title3	content3	1700-01-01
>		5	mytitle	mycontent	2025-07-29

〈그림14_DB_SQL 2_여러 필드 동시 수정 예시〉

UPDATE 활용 2

- articles 테이블 2번 레코드의 title, content 필드 값을 각각 'update Title', 'update Content' 로 변경

```
UPDATE
  articles
SET
  title = 'update Title',
  content = 'update Content'
WHERE
  id = 2;
```

※ 쿼리 실행 후 “SELECT * FROM articles;”를 통해 결과 확인

Delete data

DELETE

DELETE syntax

```
DELETE FROM table_name  
[WHERE  
    condition];
```

- DELETE FROM 절 다음에 테이블 이름 작성
- WHERE 절에서 삭제할 레코드를 지정하는 조건 작성
- WHERE 절을 작성하지 않으면 모든 레코드를 삭제



DELETE statement
테이블 레코드 삭제

<개념12_DB_SQL 2_DELETE 문 정의>

DELETE 활용 1

- articles 테이블의 1번 레코드 삭제

Q	 id INTEGER	* title VARCHAR(100)	* content VARCHAR(200)	* createdAt DATE
>	2	update Title	update Content	1900-01-01
>	3	title2	content2	1800-01-01
>	4	title3	content3	1700-01-01
>	5	mytitle	mycontent	2025-07-29

<그림15_DB_SQL 2_레코드 삭제 예시>

DELETE 활용 1

- articles 테이블의 1번 레코드 삭제

```
DELETE FROM  
  articles  
WHERE  
  id = 1;
```

- ※ 쿼리 실행 후 "SELECT * FROM articles;"를 통해 결과 확인

DELETE 활용 2

- articles 테이블에서 작성일이 오래된 순으로 레코드 2개 삭제

```
SELECT id, createdAt  
FROM articles  
ORDER BY createdAt;
```



```
DELETE FROM articles  
WHERE id = ??;
```

	id INTEC	* createdAt DATE
>	4	1700-01-01
>	3	1800-01-01
>	2	1900-01-01
>	5	2025-07-29

<그림16_DB_SQL 2_특정 필드 기준 정렬 예시>

DELETE 활용 2

- articles 테이블에서 작성일이 오래된 순으로 레코드 2개 삭제

	 id INTEGER	* title VARCHAR(100)	* content VARCHAR(200)	* createdAt DATE
>	2	update Title	update Content	1900-01-01
>	5	mytitle	mycontent	2025-07-29

<그림17_DB_SQL 2_여러 데이터 동시 삭제 예시>

DELETE 활용 2

- articles 테이블에서 작성일이 오래된 순으로 레코드 2개 삭제

```
DELETE FROM
  articles
WHERE id IN (
  SELECT id FROM articles
  ORDER BY createdAt
  LIMIT 2
);
```

- ※ 쿼리 실행 후 "SELECT * FROM articles;"를 통해 결과 확인

Multi table queries

Join

관계의 필요성 (1/4)

- 커뮤니티 게시판에 필요한 데이터 생각해보기

id	title	content	writer	role
1	제목1	내용1	하석주	admin
2	제목2	내용2	송윤미	student
3	제목3	내용3	유하선	student

<표3_DB_SQL 2_커뮤니티 게시판 테이블 예상도>



관계
여러 테이블 간의 (논리적) 연결

<개념13_DB_SQL 2_관계 정의>

관계의 필요성 (2/4)

- '하석주'가 작성한 모든 게시글을 조회하기
- 어떤 문제점이 있을까?
 - 동명이인이 있다면 혹은 특정 데이터가 수정된다면?

```
SELECT * FROM 테이블 WHERE writer = '하석주';
```

id	title	content	writer	role
1	제목1	내용1	하석주	admin
2	제목2	내용2	송윤미	student
3	제목3	내용3	유하선	student
4	제목4	내용4	하석주	student

<표4_DB_SQL2_중복 데이터 예시>

관계의 필요성 (3/4)

- 테이블을 나누어서 분류하자

articles

id	title	content
1	제목1	내용1
2	제목2	내용2
3	제목3	내용3

<표5_DB_SQL 2_articles 테이블 예시>

users

id	name
1	하석주
2	송윤미
3	유하선

<표6_DB_SQL 2_users 테이블 예시>

roles

id	role
1	admin
2	staff
3	student

<표7_DB_SQL 2_roles 테이블 예시>

- 각 게시글은 누가 작성했는지 알 수 있을까?
- 작성자들의 역할은 무엇일까?

관계의 필요성 (4/4)

- articles와 users 테이블에 각각 userId, roleId 외래 키 필드 작성

articles

id	title	content	userId
1	제목1	내용1	1
2	제목2	내용2	1
3	제목3	내용3	3

<표8_DB_SQL2_articles 테이블 예시 - user FK 추가>

users

id	name	roleId
1	하석주	1
2	송윤미	3
3	유하선	2

<표9_DB_SQL2_users 테이블 예시 role FK 추가>

roles

id	role
1	admin
2	staff
3	student

<표7_DB_SQL2_roles 테이블 예시>

관리자인 사람만 보고싶다면? → roleId가 1인 데이터 조회

하석주라는 사람이 권미숙으로 개명한다면? → users에서 한번만 변경하면 자동으로 모두 변경

JOIN이 필요한 순간

- 테이블을 분리하면 데이터 관리는 용이해질 수 있으나 출력시에는 문제가 있음
 - 테이블 한 개 만을 출력할 수 밖에 없어 다른 테이블과 결합하여 출력하는 것이 필요해짐
- 이때 사용하는 것이 'JOIN'

TIP

테이블을 나누는 이유는?

- 중복을 줄이고, 수정·삭제가 쉬워지도록 하기 위해! (예: 사용자 정보와 주문 정보는 분리)
- 하지만 화면에는?
- 사용자 이름과 주문 내용을 함께 보여줘야 하는 상황이 많음!
 - 이럴 땐 테이블을 다시 묶어야 하는데, 그 방법이 바로 JOIN

JOIN 종류

1. INNER JOIN
2. LEFT JOIN



JOIN clause

둘 이상의 테이블에서
데이터를 검색하는 방법

<개념14_DB_SQL 2_JOIN 절 정의>



사전 준비 (1/3)

- 생성하려는 테이블과 동일한 이름을 가진 테이블이 존재하는지 확인
- 불필요한 테이블이라면 DROP TABLE 명령어로 테이블 삭제
- 정상적으로 삭제되었는지 PRAGMA 활용하여 체크

```
SELECT * FROM articles;  
SELECT * FROM users;  
DROP TABLE articles;  
DROP TABLE users;  
PRAGMA table_info('articles');  
PRAGMA table_info('users');
```

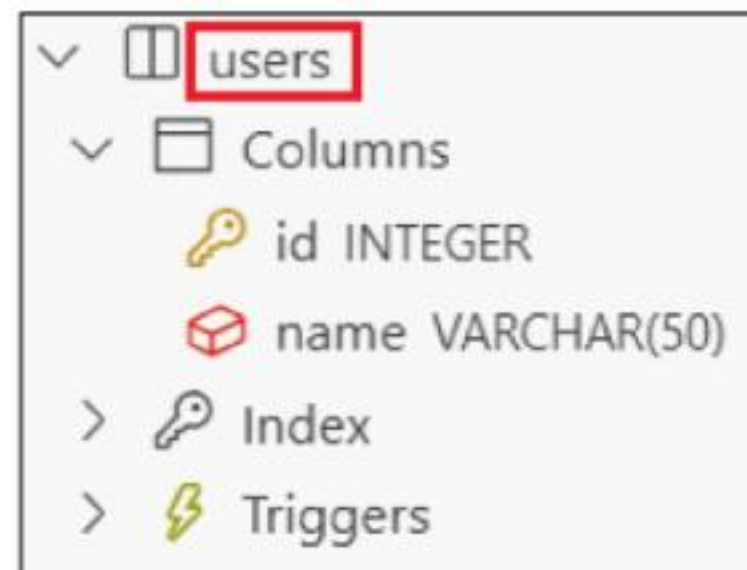
※ 동일한 결과 확인을 위해서 articles TABLE 삭제 필수!

사전 준비 (2/3)

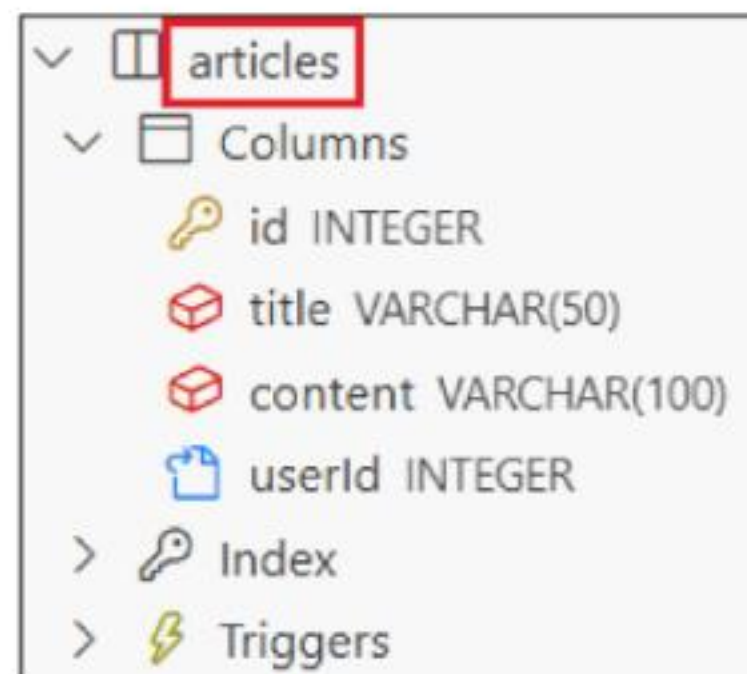
- users 및 articles 테이블 생성

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE articles (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title VARCHAR(50) NOT NULL,  
  content VARCHAR(100) NOT NULL,  
  userId INTEGER NOT NULL,  
  FOREIGN KEY (userId)  
    REFERENCES users(id)  
);
```



<그림19_DB_SQL 2_users 테이블 예시>



<그림18_DB_SQL 2_articles 테이블 예시>

사전 준비 (3/3)

- 각 테이블에 실습 데이터 입력

```
INSERT INTO  
  users (name)
```

```
VALUES
```

```
  ('하석주'),  
  ('송윤미'),  
  ('유하선');
```

```
INSERT INTO
```

```
  articles (title, content, userId)
```

```
VALUES
```

```
  ('제목1', '내용1', 1),  
  ('제목2', '내용2', 2),  
  ('제목3', '내용3', 1),  
  ('제목4', '내용4', 4),  
  ('제목5', '내용5', 1);
```

users

id	name
1	하석주
2	송윤미
3	유하선

<표10_DB_SQL 2_users 테이블 실습>

articles

id	title	content	userId
1	제목1	내용1	1
2	제목2	내용2	2
3	제목3	내용3	1
4	제목4	내용4	4
5	제목5	내용5	1

<표11_DB_SQL 2_articles 테이블 실습>

Joining tables

INNER JOIN

INNER JOIN syntax

```
SELECT  
    select_list  
FROM  
    table_a  
INNER JOIN table_b  
    ON table_b.fk = table_a.pk;
```

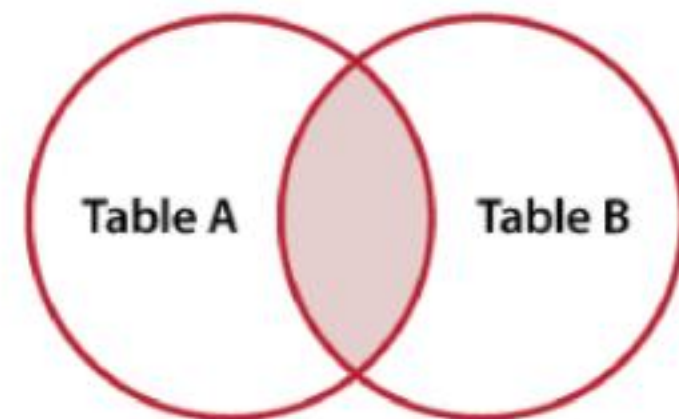
- FROM 절 이후 메인 테이블 지정 (table_a)
- INNER JOIN 절 이후 메인 테이블과 조인할 테이블을 지정 (table_b)
- ON 키워드 이후 조인 조건을 작성
- 조인 조건은 table_a와 table_b 간의 레코드를 일치시키는 규칙을 지정



INNER JOIN clause

두 테이블에서 값이
일치하는 레코드에 대해서만
결과를 반환

<개념15_DB_SQL 2 INNER JOIN 절 정의>



<그림20_DB_SQL 2 INNER JOIN 예시>

INNER JOIN 예시 (1/2)

- 작성자가 있는 (존재하는 회원) 모든 게시글을 작성자 정보와 함께 조회

articles

id	title	content	userId
1	제목1	내용1	1
2	제목2	내용2	2
3	제목3	내용3	1
4	제목4	내용4	4
5	제목5	내용5	1

<표13_DB_SQL 2_articles 테이블 실습 - INNER JOIN>

users

id	name
1	하석주
2	송윤미
3	유하선

<표12_DB_SQL 2_users 테이블 실습 - INNER JOIN>

INNER JOIN 예시 (2/2)

- 작성자가 있는 (존재하는 회원) 모든 게시글을 작성자 정보와 함께 조회

articles

id	title	content	userId
1	제목1	내용1	1
2	제목2	내용2	2
3	제목3	내용3	1
4	제목4	내용4	4
5	제목5	내용5	1

<표13_DB_SQL 2_articles 테이블 실습 - INNER JOIN>

users

id	name
1	하석주
2	송윤미
3	유하선

<표12_DB_SQL 2_users 테이블 실습 - INNER JOIN>



INNER JOIN 결과

id	title	content	userId	id	name
1	제목1	내용1	1	1	하석주
2	제목2	내용2	2	2	송윤미
3	제목3	내용3	1	1	하석주
5	제목5	내용5	1	1	하석주

<표14_DB_SQL 2_articles, users 테이블 - INNER JOIN 결과>

```
SELECT * FROM articles
INNER JOIN users
ON users.id = articles.userId;
```

INNER JOIN 활용 1

- 1번 회원(하석주)가 작성한 모든 게시글의 제목과 작성자명을 조회

	title	name
>	제목1	하석주
>	제목3	하석주
>	제목5	하석주

<그림21_DB_SQL 2_articles, users 테이블 - INNER JOIN 결과>

INNER JOIN 활용 1

- 1번 회원(하석주)가 작성한 모든 게시글의 제목과 작성자명을 조회

```
SELECT articles.title, users.name  
FROM articles  
INNER JOIN users  
  ON users.id = articles.userId  
WHERE users.id = 1;
```

	title	name
>	제목1	하석주
>	제목3	하석주
>	제목5	하석주

<그림21_DB_SQL 2_articles, users 테이블 - INNER JOIN 결과>

LEFT JOIN

LEFT JOIN syntax

```
SELECT  
    select_list  
FROM  
    table_a  
LEFT JOIN table_b  
    ON table_b.fk = table_a.pk;
```

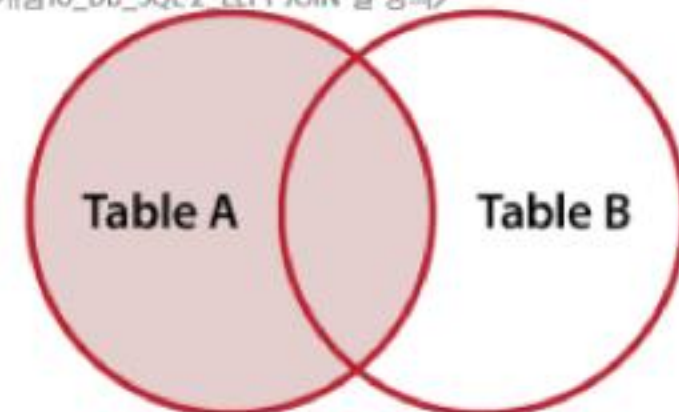
- FROM 절 이후 왼쪽 테이블 지정 (table_a)
- LEFT JOIN 절 이후 오른쪽 테이블 지정 (table_b)
- ON 키워드 이후 조인 조건을 작성
 - 왼쪽 테이블의 각 레코드를 오른쪽 테이블의 모든 레코드와 일치시킴



LEFT JOIN clause

오른쪽 테이블의 일치하는
레코드와 함께
왼쪽 테이블의 모든 레코드 반환

<개념16_DB_SQL 2 LEFT JOIN 절 정의>



<그림22_DB_SQL 2 LEFT JOIN 예시>

LEFT JOIN 예시 (1/2)

- 모든 게시글을 작성자 정보와 함께 조회

articles

id	title	content	userId
1	제목1	내용1	1
2	제목2	내용2	2
3	제목3	내용3	1
4	제목4	내용4	4
5	제목5	내용5	1

<표16_DB_SQL 2_articles 테이블 실습 - LEFT JOIN>

users

id	name
1	하석주
2	송윤미
3	유하선

<표15_DB_SQL 2_users 테이블 실습 - LEFT JOIN>

INNER JOIN 예시 (2/2)

- 작성자가 있는 (존재하는 회원) 모든 게시글을 작성자 정보와 함께 조회

articles

id	title	content	userId
1	제목1	내용1	1
2	제목2	내용2	2
3	제목3	내용3	1
4	제목4	내용4	4
5	제목5	내용5	1

<표16_DB_SQL 2_articles 테이블 실습 - LEFT JOIN>

users

id	name
1	하석주
2	송윤미
3	유하선

<표15_DB_SQL 2_users 테이블 실습 - LEFT JOIN>



INNER JOIN 결과

id	title	content	userId	id	name
1	제목1	내용1	1	1	하석주
2	제목2	내용2	2	2	송윤미
3	제목3	내용3	1	1	하석주
4	제목4	내용4	4	Null	Null
5	제목5	내용5	1	1	하석주

<표17_DB_SQL 2_articles, users 테이블 - LEFT JOIN 결과>

```
SELECT * FROM articles
LEFT JOIN users
ON users.id = articles.userId;
```

LEFT JOIN 특징

- 왼쪽 테이블의 모든 레코드를 표기
- 오른쪽 테이블과 매칭되는 레코드가 없으면 NULL을 표시

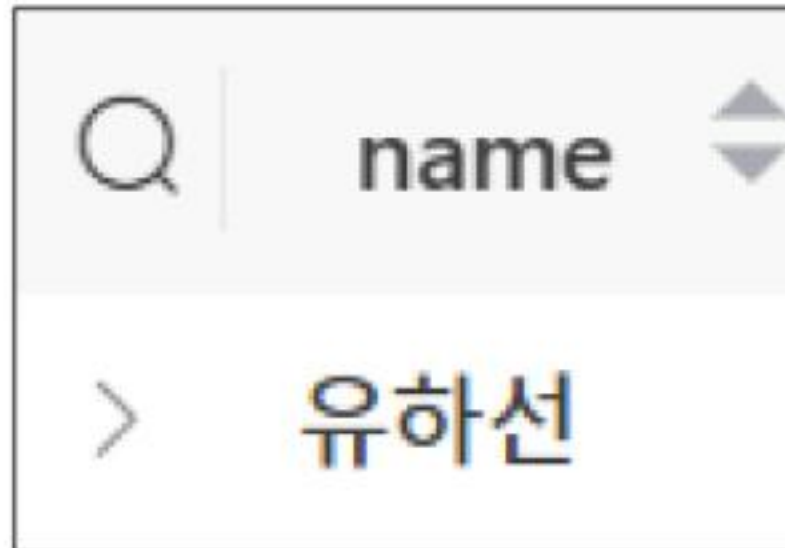
TIP

LEFT JOIN = 왼쪽 테이블 기준으로 다 보여줘!

- 왼쪽 테이블의 모든 레코드는 무조건 출력
 - 오른쪽 테이블에 연결된 값이 없으면 NULL로 표시돼요
- 왜 필요할까?
- 어떤 데이터가 없는지 확인하고 싶을 때 유용해요!
 - 예: 회원 중 아직 주문을 안 한 사람 찾기

LEFT JOIN 활용 1

- 게시글을 작성한 이력이 없는 회원 정보 조회



Q	name
>	유하선

<그림23_DB_SQL 2_articles, users 테이블 - LEFT JOIN 결과>

LEFT JOIN 활용 1

- 게시글을 작성한 이력이 없는 회원 정보 조회

```
SELECT users.name  
FROM users  
LEFT JOIN articles  
  ON articles.userId = users.id  
WHERE articles.userId IS NULL;
```

Q	name
>	유하선

<그림23_DB_SQL 2_articles, users 테이블 - LEFT JOIN 결과>

참고

타입 선호도

타입 선호도 (Type Affinity)

- 컬럼에 데이터 타입이 명시적으로 지정되지 않았거나 지원하지 않을 때 SQLite가 자동으로 데이터 타입을 추론하는 것

Example Typenames From The CREATE TABLE Statement or CAST Expression	Resulting Affinity	Rule Used To Determine Affinity
INT INTEGER TINYINT SMALLINT MEDIUMINT BIGINT UNSIGNED BIG INT INT2 INT8	INTEGER	1
CHARACTER(20) VARCHAR(255) VARYING CHARACTER(255) NCHAR(55) NATIVE CHARACTER(70) NVARCHAR(100) TEXT CLOB	TEXT	2
BLOB <i>no datatype specified</i>	BLOB	3
REAL DOUBLE DOUBLE PRECISION FLOAT	REAL	4
NUMERIC DECIMAL(10,5) BOOLEAN DATE DATETIME	NUMERIC	5

SQLite 타입 선호도의 목적

1. 유연한 데이터 타입 지원

- 데이터 타입을 명시적으로 지정하지 않고도 데이터를 저장하고 조회할 수 있음
- 컬럼에 저장되는 값의 특성을 기반으로 데이터 타입을 유추

2. 간편한 데이터 처리

- INTEGER Type Affinity를 가진 열에 문자열 데이터를 저장해도 SQLite는 자동으로 숫자로 변환하여 처리

3. SQL 호환성

- 다른 데이터베이스 시스템과 호환성을 유지

NOT NULL

반드시 NOT NULL 제약을 사용해야 할까?

- “NO”
- 하지만 데이터베이스를 사용하는 프로그램에 따라 NULL을 저장 할 필요가 없는 경우가 많으므로 대부분 NOT NULL을 정의
- “값이 없다.”는 표현을 테이블에 기록하는 것은 “0”이나 “빈 문자열” 등을 사용하는 것으로 대체하는 것을 권장



Constraints

테이블의 필드에 적용되는
규칙 또는 제한 사항

<개념2_DB_SQL 2_ 제약 조건 정의>

날짜와 시간

SQLite의 날짜와 시간

- SQLite에는 날짜 및/또는 시간을 저장하기 위한 별도 데이터 타입이 없음
- 대신 날짜 및 시간에 대한 함수를 사용해 표기 형식에 따라 TEXT, REAL, INTEGER 값으로 저장
- <https://www.sqlite.org/datatype3.html>

SQL 총 정리

- 1894. 주문 정보 DB 관리 - 테이블 생성과 데이터 삽입
- 1895. 주문 정보 DB 관리 - 테이블 삭제와 JOIN
- 3052. SQL 조회 - artists
- 3053. SQL 조회 - invoices 1
- 3054. SQL 조회 - invoices 2
- 2906. 동물원 DB 관리 - 복잡한 조회
- 2907. 동물원 DB 관리 - 데이터 수정, 삭제
- 2908. 호텔 객실 DB 관리
- 2909. 유저 정보 DB 관리
- 3278. 직원 정보 DB 관리 - 집계 함수

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 테이블을 생성하는 SQL 문으로 올바른 것은?

- a) INSERT TABLE users (...);
- b) CREATE DATABASE users;
- c) CREATE TABLE users (...);
- d) SELECT TABLE users (...);

2 테이블에서 name과 age 컬럼만 선택적으로 조회하고자 할 때 올바른 SQL 문은?

- a) SELECT ALL name, age FROM users;
- b) FIND name, age IN users;
- c) SELECT name, age FROM users;
- d) SHOW name AND age FROM users;

3 다음 중 UPDATE 문에 대한 설명으로 올바른 것은?

- a) 테이블 전체를 삭제한다.
- b) 특정 데이터를 수정한다.
- c) 테이블 구조를 수정한다.
- d) 새로운 데이터를 삽입한다.

4 다음 중 외래 키(Foreign Key)의 기능으로 가장 적절한 것은?

- a) 다른 테이블과의 관계를 설정하는 기능
- b) 데이터를 정렬하는 기능
- c) 테이블을 삭제하는 기능
- d) 중복된 데이터를 제거하는 기능

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 ALTER TABLE 명령어로 할 수 없는 작업은 무엇인가?

- a) 새로운 컬럼 추가
- b) 기존 컬럼 이름 변경
- c) 기존 컬럼 삭제
- d) 전체 테이블 삭제

7 DELETE FROM users WHERE id = 1; 명령어의 기능은 무엇인가?

- a) 모든 레코드를 삭제한다.
- b) id가 1인 레코드만 삭제한다.
- c) 테이블을 삭제한다.
- d) 새로운 레코드를 추가한다

6 다음 SQL 문 중 오름차순으로 정렬된 결과를 얻기 위한 구문은?

- a) SELECT * FROM users DESC;
- b) ORDER BY users ASC SELECT *;
- c) SELECT * FROM users ORDER BY age ASC;
- d) SORT users BY age;

8 SELECT DISTINCT country FROM customers; 문장의 목적은?

- a) 국가명이 'country'인 행만 선택
- b) 모든 필드를 중복 포함 없이 조회
- c) 중복 없이 고유한 국가명만 조회
- d) 국가명이 없는 레코드를 제거

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 중 기본 키(Primary Key)의 특징으로 가장 알맞은 것은?

- a) 중복된 값을 허용한다.
- b) NULL 값을 여러 개 가질 수 있다.
- c) 두 개 이상의 테이블을 정렬한다.
- d) 각 레코드를 고유하게 식별한다.

11 다음 중 GROUP BY 절에 대한 설명으로 바른 것은?

- a) 레코드를 삭제하는 데 사용
- b) 특정 필드를 기준으로 데이터를 그룹화
- c) 테이블을 복사하는 데 사용
- d) 테이블 이름을 변경하는 데 사용

10 SQL에서 LIKE '_a%' 조건의 의미는?

- a) 두 번째 글자가 'a'로 시작하는 문자열
- b) 'a'로 시작하는 문자열
- c) 'a'로 끝나는 세 글자 문자열
- d) 'a'가 포함된 모든 문자열

12 DROP TABLE employees; 명령이 실행되면 어떤 일이 발생하는가?

- a) employees 테이블의 구조만 초기화
- b) employees 테이블이 삭제됨
- c) employees 테이블에서 모든 데이터만 삭제됨
- d) employees 테이블의 이름이 변경됨

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

- 13 다음 SQL 문에 대한 설명으로 가장 적절한 것은 무엇인가요?

```
UPDATE customers  
SET Country = 'Korea'  
WHERE Country = 'KR';
```

- a) 고객 테이블에서 Country가 'KR'인 행만 'Korea'로 수정한다.
- b) 고객 테이블에서 모든 Country 값을 'Korea'로 일괄 변경한다.
- c) 고객 테이블에서 'Korea' 값을 가진 행을 삭제한다.
- d) 고객 테이블에서 Country가 'Korea'인 행을 모두 'KR'로 변경한다.

- 14 다음 SQL 문이 반환하는 결과로 올바른 것을 고르세요.

```
SELECT department, COUNT(*) AS cnt  
FROM employees  
GROUP BY department;
```

- a) 각 부서의 직원 수를 구하여 부서 이름과 함께 보여준다.
- b) 부서별로 평균 급여를 계산한다.
- c) 직원들의 이름을 알파벳 순서로 정렬한 목록을 보여준다.
- d) employees 테이블 전체를 그대로 조회한다

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

15 다음 SQL 문은 어떤 레코드를 반환할까요?

```
SELECT *  
FROM users  
WHERE name LIKE 'Lee%';
```

- a) 이름이 'Lee'로 끝나는 사용자만 조회
- b) 이름에 'Lee'가 포함된 모든 사용자 조회
- c) 이름이 정확히 'Lee'인 사용자만 조회
- d) 이름이 'Lee'로 시작하는 사용자 모두 조회

16 다음 SQL 문은 어떤 작업을 수행하나요?

```
ALTER TABLE products  
ADD COLUMN stock  
INTEGER NOT NULL DEFAULT 0;
```

- a) products 테이블에서 stock 열을 삭제한다.
- b) stock 컬럼의 값을 모두 0으로 덮어쓴다.
- c) products 테이블에 stock 컬럼을 새로 추가하고 기본값을 0으로 지정한다.
- d) stock 컬럼의 이름을 변경한다

확인 문제



정답 및 해설

1) c) CREATE TABLE users (...); 2) c) SELECT name, age FROM users; 3) b) 특정 데이터를 수정한다. 4) a) 다른 테이블과의 관계를 설정하는 기능
5) d) 전체 테이블 삭제 6) c) SELECT * FROM users ORDER BY age ASC; 7) b) id가 1인 레코드만 삭제한다. 8) c) 중복 없이 고유한 국가명만 조회
9) d) 각 레코드를 고유하게 식별한다. 10) a) 두 번째 글자가 'a'로 시작하는 문자열 11) b) 특정 필드를 기준으로 데이터를 그룹화
12) b) employees 테이블이 삭제됨

1. CREATE TABLE은 테이블을 생성하는 SQL 명령어이며, 괄호 안에 컬럼과 타입을 정의합니다.
2. SELECT [컬럼명] FROM [테이블명] 문법이 기본 SELECT 문 구조입니다.
3. UPDATE는 조건에 맞는 기존 데이터를 수정하는 SQL 명령어입니다.
4. 외래 키는 두 테이블 간의 관계를 연결하기 위해 사용되며, 주로 다른 테이블의 기본 키를 참조합니다.
5. 전체 테이블 삭제는 DROP TABLE 문으로 수행합니다. ALTER TABLE은 테이블 구조 변경용입니다.
6. ORDER BY [컬럼] ASC는 오름차순 정렬 구문입니다.

7. DELETE ... WHERE는 조건에 맞는 특정 레코드를 삭제합니다. 전체 삭제는 WHERE 없이 사용.
8. DISTINCT는 중복을 제거하여 고유한 값만 조회합니다.
9. 기본 키는 테이블의 각 행을 유일하게 식별하며 중복과 NULL을 허용하지 않습니다.
10. _는 한 글자, %는 0개 이상의 문자. 따라서 두 번째 글자가 'a'인 패턴입니다.
11. GROUP BY는 데이터를 특정 열 기준으로 묶어 집계할 때 사용합니다.
12. DROP TABLE은 테이블 전체(구조 + 데이터)를 삭제합니다.

확인 문제



정답 및 해설

13) d) 고객 테이블에서 Country가 'Korea'인 행을 모두 'KR'로 변경한다. 14) a) 각 부서의 직원 수를 구하여 부서 이름과 함께 보여준다. 15) d) 이름이 'Lee'로 시작하는 사용자 모두 조회 16) c) products 테이블에 stock 컬럼을 새로 추가하고 기본값을 0으로 지정한다.

13. 이 쿼리는 UPDATE 문을 사용하여 특정 조건을 만족하는 행의 값을 수정하는 예시입니다.

- UPDATE customers는 customers 테이블의 레코드를 수정하겠다는 의미입니다.
- SET Country = 'Korea'는 수정할 컬럼과 새로운 값을 지정합니다.
- 즉, Country 컬럼을 'Korea'로 바꾸겠다는 뜻입니다.

14. 이 쿼리는 집계 함수와 GROUP BY 절을 함께 사용하는 전형적인 그룹 집계 쿼리입니다.

- SELECT department, COUNT(*) AS cnt: 부서명(department)과 해당 부서에 속한 직원 수(COUNT(*))를 조회하라는 뜻입니다.
- AS cnt는 별칭으로, 결과 컬럼 이름을 cnt로 지정합니다.
- FROM employees: 조회할 테이블은 employees입니다. GROUP BY department: 같은 department 값끼리 그룹을 만든 뒤, 각 그룹에 대해 집계함수를 적용합니다.

15. 'Lee%'는 Lee로 시작하고 뒤에 어떤 문자가 와도 상관없다는 의미입니다. %는 0개 이상의 임의 문자와 일치합니다.

- a) 'Lee%'는 시작 조건이지 끝 조건이 아닙니다. → 끝나는 조건은 '%Lee'
- b) 'Lee%'는 포함이 아닌 시작 조건입니다. 포함 조건은 '%Lee%'
- c) 정확히 'Lee'만 조회하려면 WHERE name = 'Lee'로 작성해야 합니다.

16. 이 명령은 products 테이블에 새로운 정수형 컬럼 stock을 추가하고, 기존 레코드에는 기본값 0을 자동으로 채우게 됩니다.

- ALTER TABLE ... ADD COLUMN 구문은 테이블에 새 컬럼을 추가할 때 사용됩니다.
- stock INTEGER은 stock이라는 정수형 컬럼을 추가하겠다는 의미이며, NOT NULL은 이 컬럼이 NULL 값을 가질 수 없음을 뜻합니다.
- DEFAULT 0은 값을 지정하지 않아도 기본적으로 0이 입력되도록 설정합니다.

핵심 키워드

개념	설명	예시
CREATE TABLE	새로운 테이블을 생성하는 SQL 명령어	CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT);
INSERT	테이블에 새로운 데이터를 삽입하는 명령어	INSERT INTO users (name) VALUES ('하석주');
SELECT	테이블에서 데이터를 조회하는 명령어	SELECT * FROM users;
UPDATE	기존 데이터를 수정하는 명령어	UPDATE users SET name = '김민수' WHERE id = 1;
DELETE	테이블의 데이터를 삭제하는 명령어	DELETE FROM users WHERE id = 1;
ALTER TABLE	테이블 구조를 변경하는 명령어	ALTER TABLE users ADD COLUMN age INTEGER;
DROP TABLE	테이블 자체를 삭제하는 명령어	DROP TABLE users;
PRIMARY KEY	테이블에서 각 행을 고유하게 식별하는 열 지정	id INTEGER PRIMARY KEY

활동 정리

핵심 키워드

개념	설명	예시
NOT NULL	해당 열에 반드시 값이 있어야 함을 나타내는 제약조건	name TEXT NOT NULL
FOREIGN KEY	다른 테이블의 값을 참조하는 제약조건	userId INTEGER REFERENCES users(id)
AUTOINCREMENT	새로운 행이 추가될 때 자동으로 숫자를 증가시키는 속성	id INTEGER PRIMARY KEY AUTOINCREMENT
INNER JOIN	두 테이블에서 일치하는 데이터만 조회	SELECT * FROM articles INNER JOIN users ON users.id = articles.userId;
LEFT JOIN	왼쪽 테이블의 모든 행을 유지하며 일치하는 데이터를 오른쪽 테이블에서 조회	SELECT * FROM users LEFT JOIN articles ON users.id = articles.userId;
PRAGMA	테이블의 구조 등 메타데이터 정보를 확인하는 SQLite 명령어	PRAGMA table_info('users');

활동 정리

오늘 공부한 내용 요약 및 정리

- CREATE TABLE 문을 통해 새로운 테이블을 만들고, 각 열에 데이터 타입과 제약 조건을 설정할 수 있다.
- 제약 조건(Constraints)은 데이터의 무결성과 일관성을 유지하기 위해 사용되며, PRIMARY KEY, NOT NULL, FOREIGN KEY 등이 있다.
- INSERT / SELECT / UPDATE / DELETE 문을 활용해 테이블에 데이터를 추가, 조회, 수정, 삭제할 수 있다.
- ALTER TABLE 문을 사용하면 테이블에 열을 추가하거나 이름을 변경하고, 열을 삭제하거나 테이블 이름을 바꿀 수 있다.
- DROP TABLE 명령은 테이블 자체를 삭제할 때 사용된다.
- JOIN을 사용하면 분리된 여러 테이블을 조합하여 하나의 결과로 조회할 수 있다.
- INNER JOIN은 양쪽 모두에 데이터가 있는 경우만, LEFT JOIN은 왼쪽 기준으로 모두 보여준다.
- SQLite는 데이터 타입에 유연하며, 명시적 타입이 없어도 데이터를 저장하고 추론할 수 있는 타입 선호도(Type Affinity)를 가진다.
- 날짜와 시간은 별도 타입 없이 TEXT, REAL, INTEGER 타입으로 저장하며, 함수로 처리한다.

활동 정리

"나만의 작은 도서관, 직접 만들어 볼까요?"

- 집에 있는 책을 정리해서 나만의 도서관을 만든다고 상상해 보세요!
 - 이제 어떤 책이 있는지, 누가 빌려갔는지, 언제 샀는지까지 정리해보려면 어떻게 해야 할까요?

1. 책장 하나를 {{ CREATE TABLE }} 문으로 만들고,
2. 책 정보를 {{ INSERT }} 문으로 채워 넣습니다.
3. "2020년 이후에 산 책"만 {{ SELECT }} 문으로 골라볼 수도 있죠.

- SQL은 데이터를 정리하는 책장 같은 도구입니다.
- 원하는 데이터를 빠르게 찾을 수 있도록 도와줍니다.
- 스스로 데이터를 만들고, 저장하고, 찾아보는 과정을 직접 체험할 수 있습니다.

감사합니다

